

CNN (Convolutional Neural Network) – Documentation

1. What is CNN?

A **Convolutional Neural Network (CNN)** is a deep learning algorithm mainly used for **image processing and classification**. It automatically learns important features from images like edges, shapes, and textures without manual feature extraction.

CNN is widely used in:

- Image classification
 - Face recognition
 - Object detection
 - Medical image analysis
-

2. CNN Architecture Overview

A typical CNN consists of:

◆ 1. Input Layer

- Takes image input (e.g., $64 \times 64 \times 3$ for RGB images)
-

◆ 2. Convolution Layer

- Applies filters (kernels) to extract features
- Detects edges, patterns, textures

👉 Operation:

- Sliding filter over image
 - Produces **feature maps**
-

◆ 3. Activation Function (ReLU)

- Introduces non-linearity
- Removes negative values

👉 Formula:
 $\text{ReLU}(x) = \max(0, x)$

◆ 4. Pooling Layer

- Reduces size of feature maps
- Keeps important features
- Improves computation speed

Types:

- Max Pooling (most common)
 - Average Pooling
-

◆ 5. Flatten Layer

- Converts 2D feature maps into 1D vector
-

◆ 6. Fully Connected Layer (Dense Layer)

- Performs classification
 - Combines extracted features
-

◆ 7. Output Layer

- Uses Softmax/Sigmoid activation
 - Produces final prediction
-



3. CNN Working Flow

1. Image input
2. Convolution → feature extraction
3. ReLU → activation
4. Pooling → downsampling
5. Repeat layers
6. Flatten
7. Dense layers

8. Output prediction

4. CNN Implementation Code (TensorFlow/Keras)

Step 1: Import Libraries

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

Step 2: Load Dataset

(Example: dataset folder structure required)

```
train_datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_data = train_datagen.flow_from_directory(
    "dataset/",
    target_size=(64, 64),
    batch_size=32,
    class_mode='binary',
    subset='training'
)

val_data = train_datagen.flow_from_directory(
    "dataset/",
    target_size=(64, 64),
    batch_size=32,
    class_mode='binary',
    subset='validation'
)
```

Step 3: Build CNN Model

```
model = Sequential()

# Convolution Layer 1
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(64,64,3)))
model.add(MaxPooling2D(pool_size=(2,2)))

# Convolution Layer 2
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
```

```
# Convolution Layer 3
model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

# Flatten
model.add(Flatten())

# Fully Connected Layer
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))

# Output Layer
model.add(Dense(1, activation='sigmoid'))
```

Step 4: Compile Model

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

Step 5: Train Model

```
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=10
)
```

Step 6: Save Model

```
model.save("cnn_model.h5")
```

Step 7: Load Model (for testing)

```
from tensorflow.keras.models import load_model

model = load_model("cnn_model.h5")
```

5. Prediction Code

```
import numpy as np
from tensorflow.keras.preprocessing import image

img = image.load_img("test.jpg", target_size=(64,64))
```

```
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0) / 255.0

prediction = model.predict(img_array)

if prediction[0][0] > 0.5:
    print("Class 1")
else:
    print("Class 0")
```

6. Streamlit Deployment Code (app.py)

```
import streamlit as st
import numpy as np
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image

model = load_model("cnn_model.h5")

st.title("CNN Image Classifier")

uploaded_file = st.file_uploader("Upload an Image")

if uploaded_file is not None:
    img = image.load_img(uploaded_file, target_size=(64,64))
    st.image(img, caption="Uploaded Image")

    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0

    prediction = model.predict(img_array)

    if prediction[0][0] > 0.5:
        st.write("Prediction: Class 1")
    else:
        st.write("Prediction: Class 0")
```